Tema 2

“Búsqueda no-informada”

Año Académico 2026/27

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

- Espacio de estados
- Algoritmos de búsqueda no informados

Tema 3: Búsqueda heurística

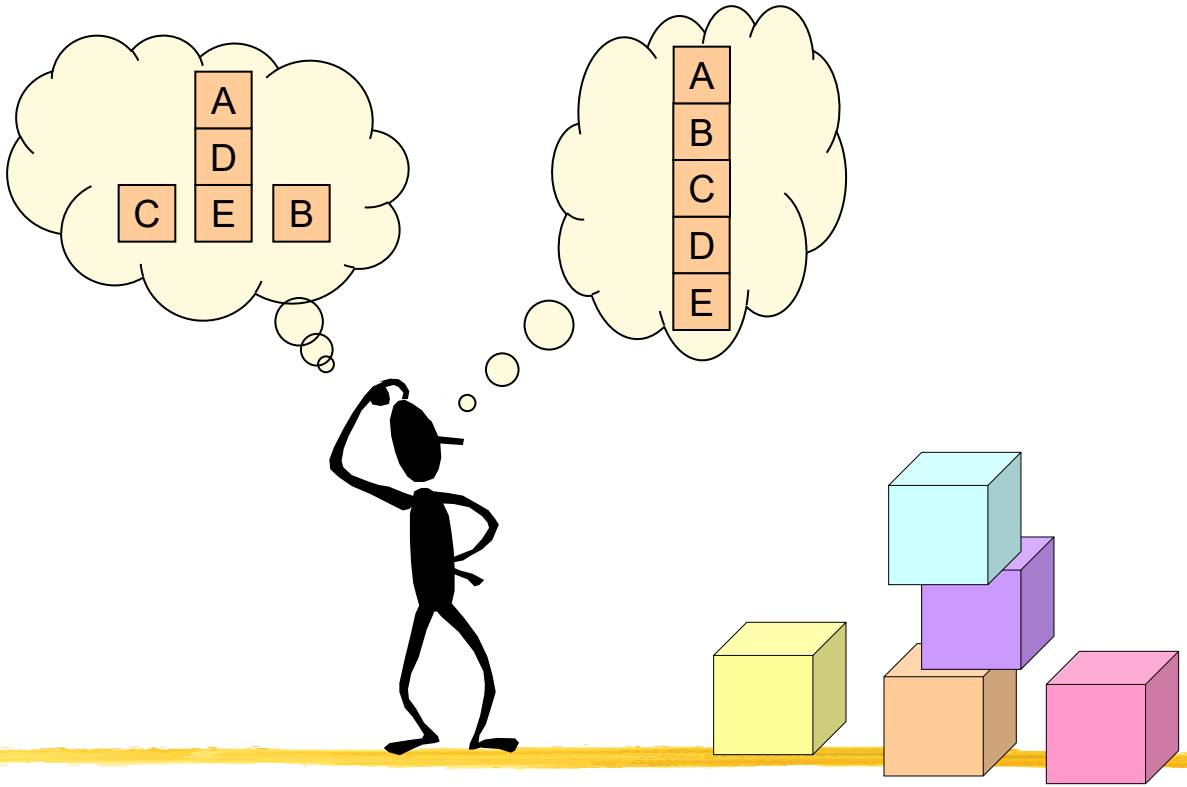
Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

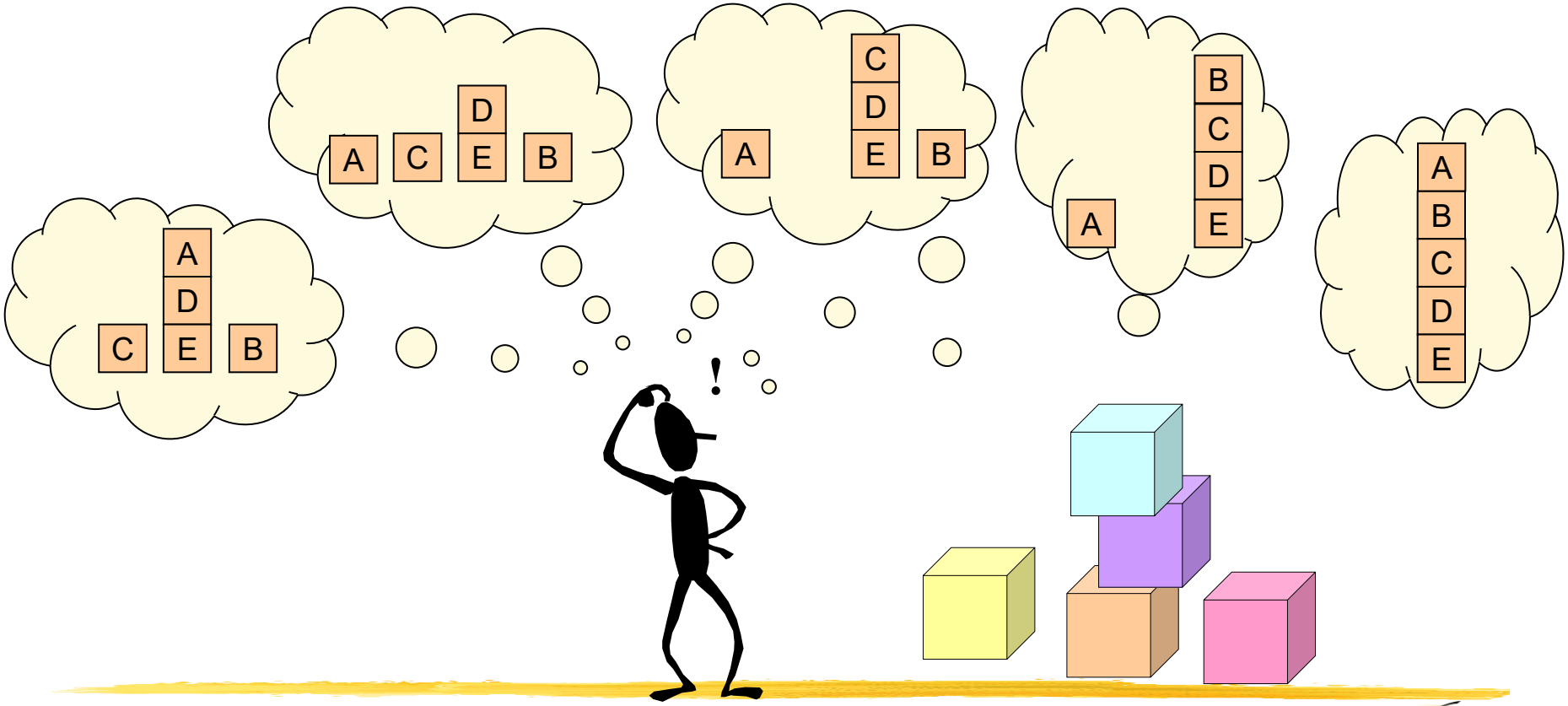
Agentes basados en búsqueda

- Son capaces de percibir el **estado actual** del mundo (la situación)
- Tienen un determinado **objetivo/meta**: "cómo debería ser el mundo"
- Son capaces de realizar **acciones** que **modifican** el mundo
- **Buscan** una secuencia de acciones que modifiquen el mundo para llegar a su objetivo



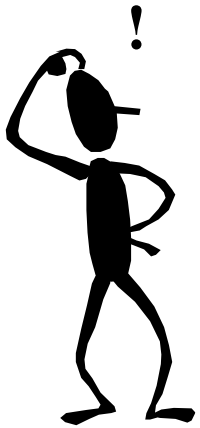
Agentes basados en búsqueda

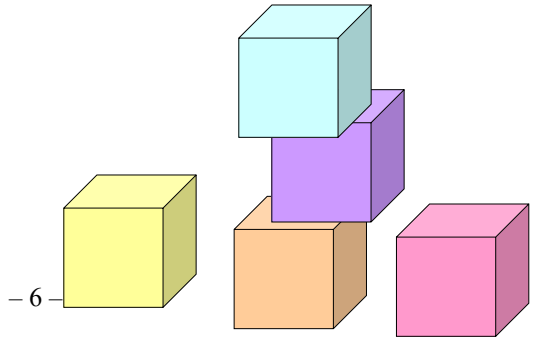
- Con tal fin:
  - Mantienen un **modelo *simbólico*** del entorno
  - *anticipan* los efectos esperados de sus acciones sobre el modelo,
  - generando y seleccionando ***planes de actuación*** en el modelo…



Agentes basados en búsqueda

– … antes de ejecutar las acciones del plan en el entorno real





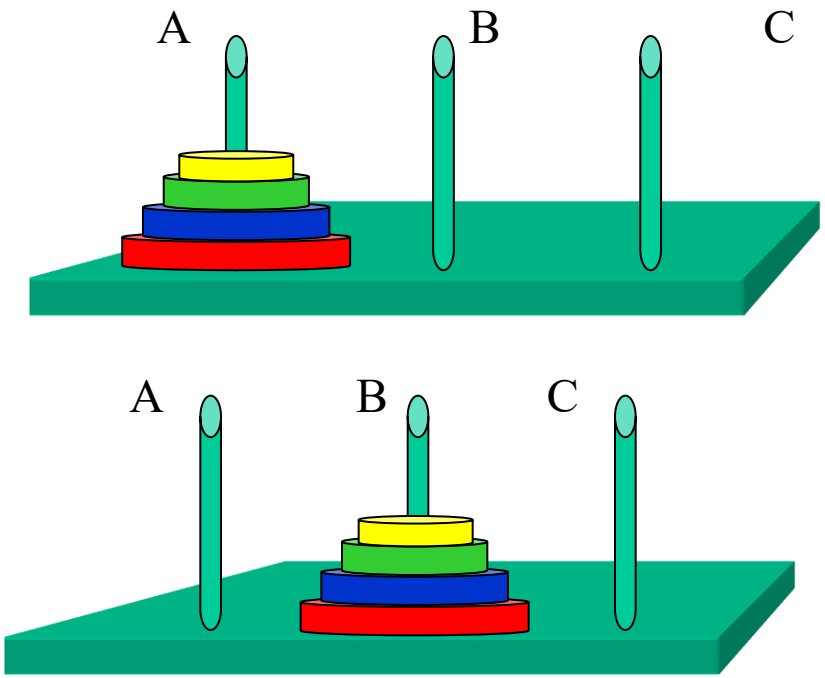
Entornos simples

Características de entornos simples (aka: *problemas bien definidos*)

- discreto:
  - se puede concebir el mundo en estados
  - en cada estado hay un conjunto finito de percepciones y acciones
- accesible: el agente puede acceder a las características relevantes del entorno
  - puede determinar el estado actual del mundo
  - puede determinar el estado del mundo que le gustaría alcanzar
- estático y determinista: no hay presión temporal ni incertidumbre
  - el mundo cambia sólo cuando el agente actúa
  - el resultado de cada acción está totalmente definido y previsible

Búsqueda como método genérico

Ejemplo: Torres de Hanoi



Objetivo:

- Trasladar los discos de la aguja *A* a *B* en el mismo orden

Restricción:

- un disco mayor nunca debe reposar sobre uno de menor tamaño

¿Cómo escribir el programa de agente correspondiente?

Solución 1: tablas de actuación

Tablas de actuación específicos del problema:

- para cada situación hay una entrada en una tabla de actuación; dicha entrada compila la secuencia de acciones a emprender:

  cuatro discos en A $\Rightarrow$

  disco 1 de A a C / disco 2 de A a B / disco 1 de C a B / disco 3 de A a C / disco 1 de B a A / disco 2 de B a C /disco 1 de A a C / disco 4 de A a B / disco 1 de C a B / disco 2 de C a A /disco 1 de B a A/ disco 3 de C a B / disco 1 de A a C / disco 2 de A a B/ disco 1 de C a B

- mejorar la flexibilidad: *aprender* nuevas entradas
- problema: limitaciones de memoria

Solución 2: algoritmo

Algoritmos específicos del problema:

- el diseñador del agente conoce un método para resolver el problema
- codifica este método en un algoritmo *particular para el problema*
- mejorar la flexibilidad: *parametrizar* el algoritmo
- problema: el diseñador ha de *anticipar* todos los escenarios posibles
- los entornos reales suelen ser demasiado complejos como para anticipar todas las posibilidades

```
PROCEDURE MoverDiscos(n:integer;
                      origen,destino,auxiliar:char);
{ Pre: n > 0
  Post: output = [movimientos para pasar n
                  discos de la aguja origen
                   a la aguja destino] }
BEGIN
  IF n = 0 THEN   {Caso base}
    writeln
  ELSE BEGIN      {Caso recurrente}
    MoverDiscos(n-1,origen,auxiliar,destino);
    write('Pasar disco',n,'de',origen,'a',destino);
    MoverDiscos(n-1,auxiliar,destino,origen)
  END; {fin ELSE}
END; {fin MoverDiscos}
```

Solución 3: búsqueda

Métodos independientes del problema :

- modelo simbólico del problema:
  - "inicialmente todos los discos reposan en *A* y su tamaño decrece de abajo hasta arriba"
  - "queremos que todos los discos estén en *C* en el mismo orden"
  - "podemos mover un disco *I* a la aguja *X*, si no hay otro disco por encima de *I* y, si actualmente hay discos en *X*, entonces dichos discos han de ser más grandes que *I*"
  - "cuanto menos movimientos de discos hagamos mejor"
- algoritmo de búsqueda *genérico*:
  - genera una solución a cualquier problema representado mediante el modelo simbólico
- mayor flexibilidad:
  - el diseñador no necesita conocer la solución de antemano
  - es más fácil adaptar el método a nuevas características del problema

Búsqueda en espacios de estados

Espacio de estados: modelo del mundo representado por un grafo

mundo	modelo	representación
situacion	estado	nodo
situación presente	estado inicial	nodo inicial
acción y sus efectos	operador	arco
secuencia de acciones	plan	camino

Problema de búsqueda: espacio de estados + actitud del agente

actitud	representación
estados meta	conjunto de nodos
eficiencia de un plan	coste de un camino

Objetivo: encontrar el plan más eficiente que lleve del estado inicial a un estado meta

Ejemplo: El mundo de los bloques

Situaciones:

- *n* bloques en una mesa de longitud ilimitada

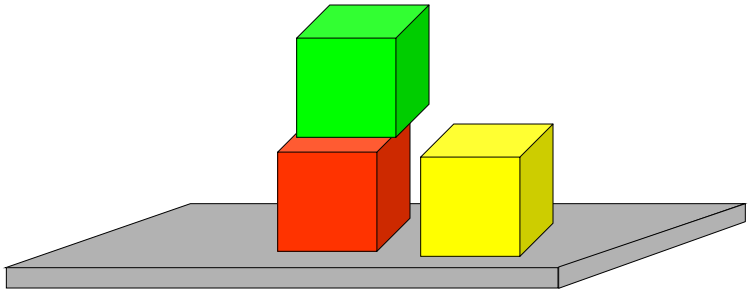
Acciones:

- *apilar*(*X*,*Y*): poner *X* encima de *Y*
  - Prec.: bloques *X* e *Y* están libres
  - Efecto: bloque *X* está encima de *Y*
- *quitar*(*Y*): poner *Y* en la mesa
  - Prec.: bloque *Y* está libre
  - Efecto: bloque *Y* está en la mesa

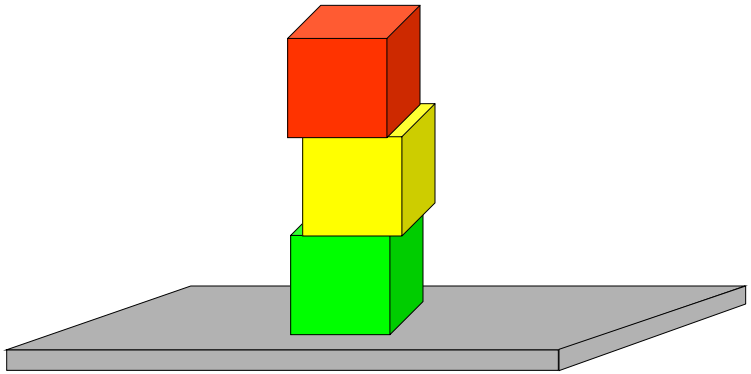
Actitud del agente:

- Objetivo: cierta configuración de bloques
- Sólo la posición *vertical* es relevante
- El coste de cada acción es uno

Situación presente

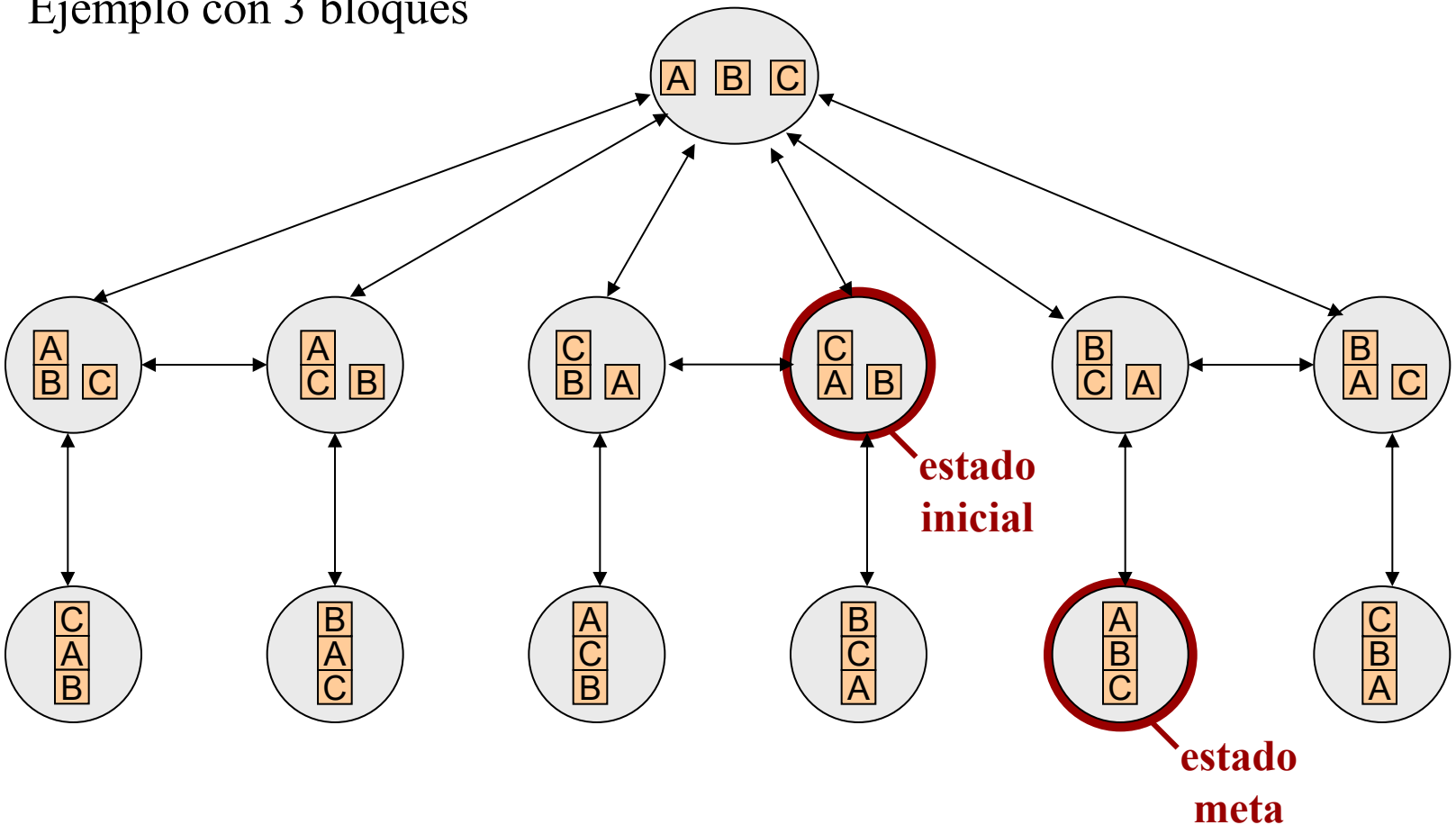


Objetivo



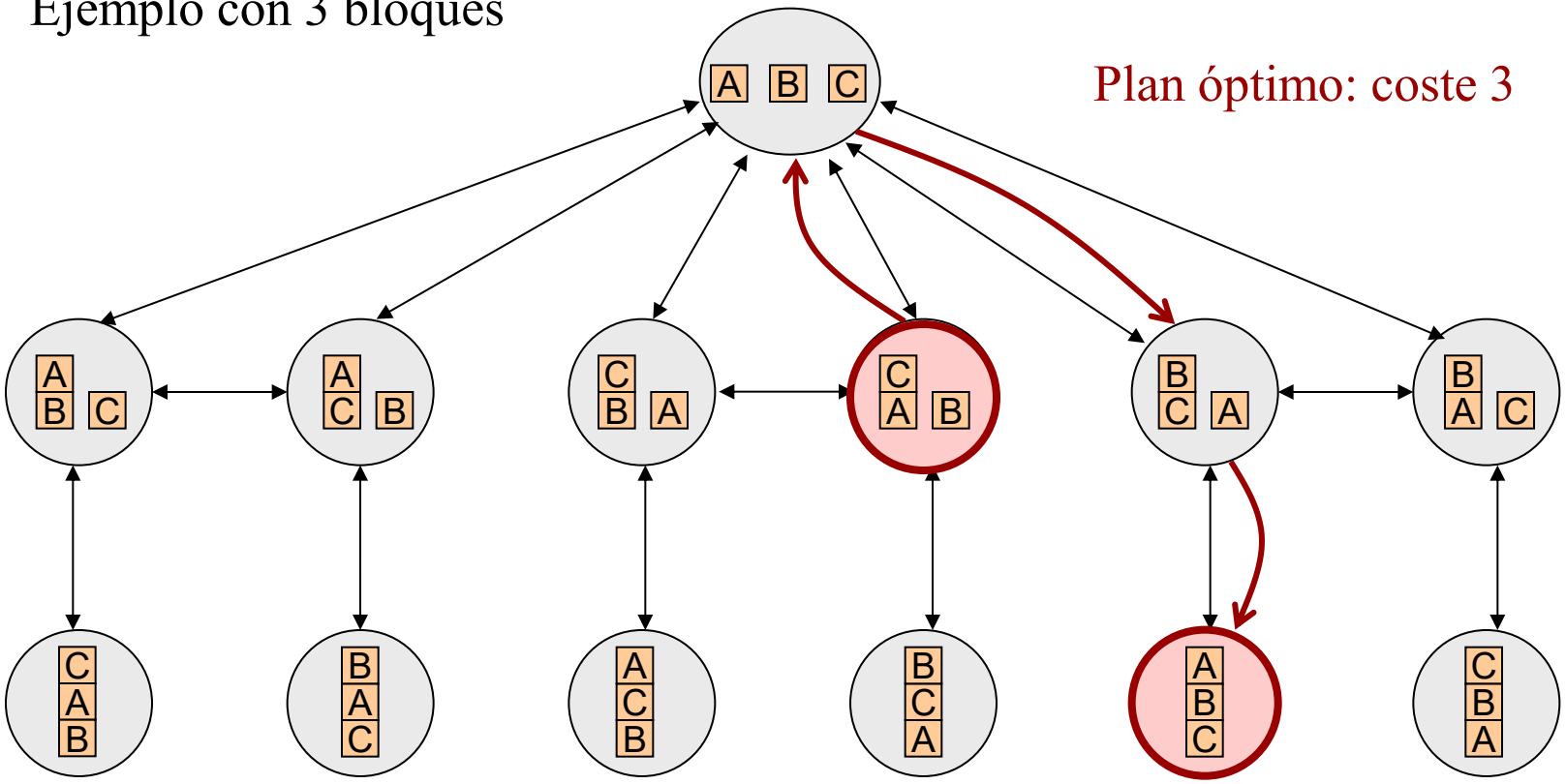
Mundo de los bloques: espacio de estados

Ejemplo con 3 bloques



Representación del problema de los bloques

Ejemplo con 3 bloques



Plan óptimo: coste 3

Agentes especializados

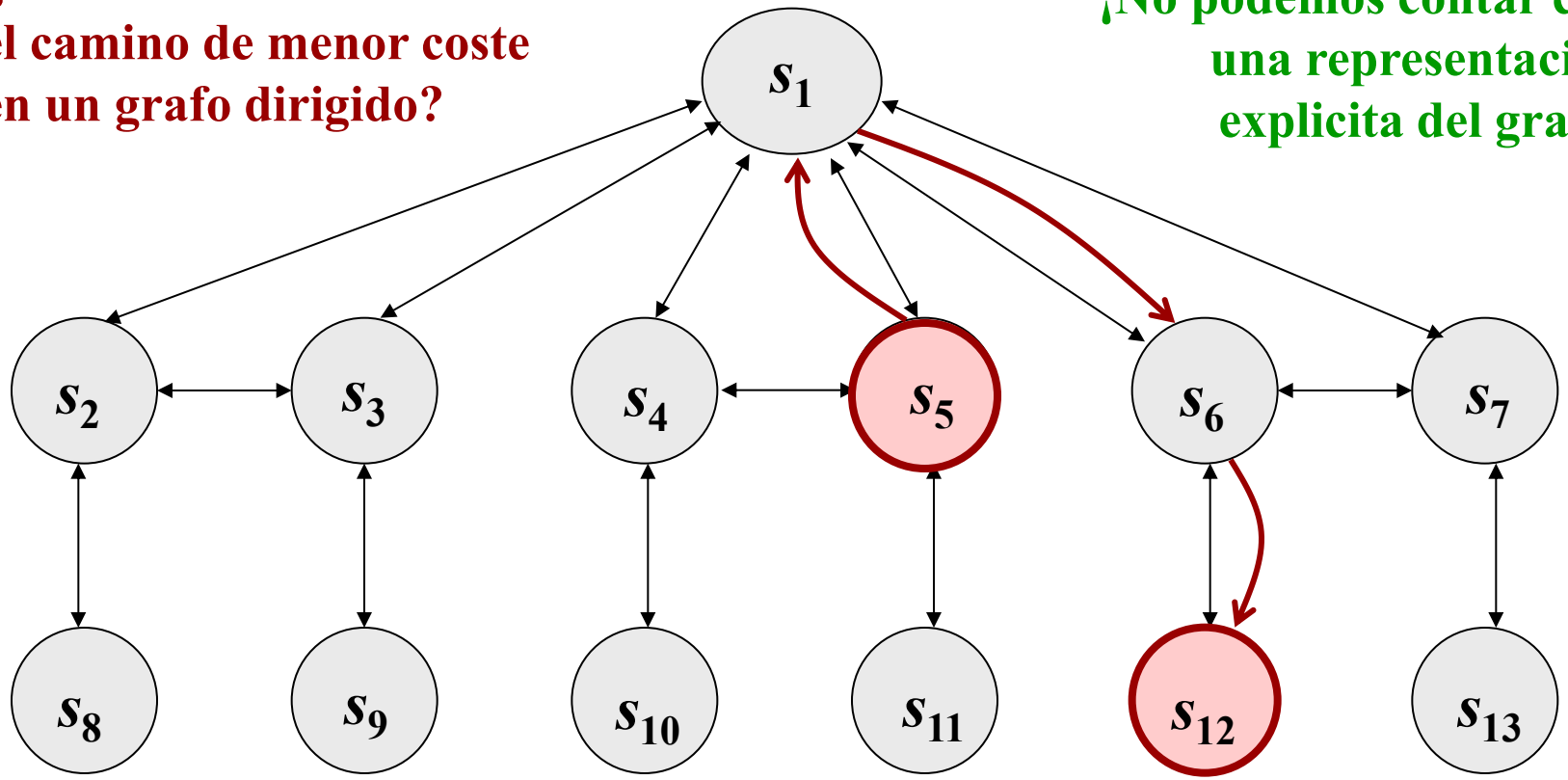
Tareas:

1. Definir el *modelo* y generar los *objetivos*
   - Conocimiento a priori del agente facilitado por el diseñador
2. Generar el espacio de estados:
   - ver Módulo 3 (y, como introducción, Tema 6 en este módulo)
3. *Percibir* y *clasificar* la situación presente:
   - Agentes hardware: Visión artificial, Robótica
   - Agentes software: Algoritmos específicos del dominio
4. *Buscar* un *plan* de actuación
5. *Ejecutar* el plan de actuación
   - Agentes hardware: Robótica
   - Agentes software: Algoritmos específicos del dominio

Búsqueda en espacio de estados

¿Se trata "sólo" de encontrar el camino de menor coste en un grafo dirigido?

¡No podemos contar con una representación explicita del grafo!



Conocimientos “a priori” de nuestro agente

- Representación *implícita* del problema de búsqueda
- Conocimientos mínimos a priori de un agente:
  - s_0 — Estado *inicial*
  - *expandir*: $s \mapsto \{s_{i_1}, ..., s_{i_n}\}$ — Conjunto finito de *sucesores* de un estado
  - *meta?*: s $\mapsto$ verdad | falso — Prueba de *éxito* en un estado
  - c: $(s_i, s_j) \mapsto v$, $v \in \aleph$ — *Coste* de un operador
  - $c(s_{i_1}\ s_{i_2} \ldots s_{i_n}) = \sum_{k=1}^{n-1} c(s_{i_k}, s_{i_{k+1}})$ — *Coste* de un plan

Método de búsqueda

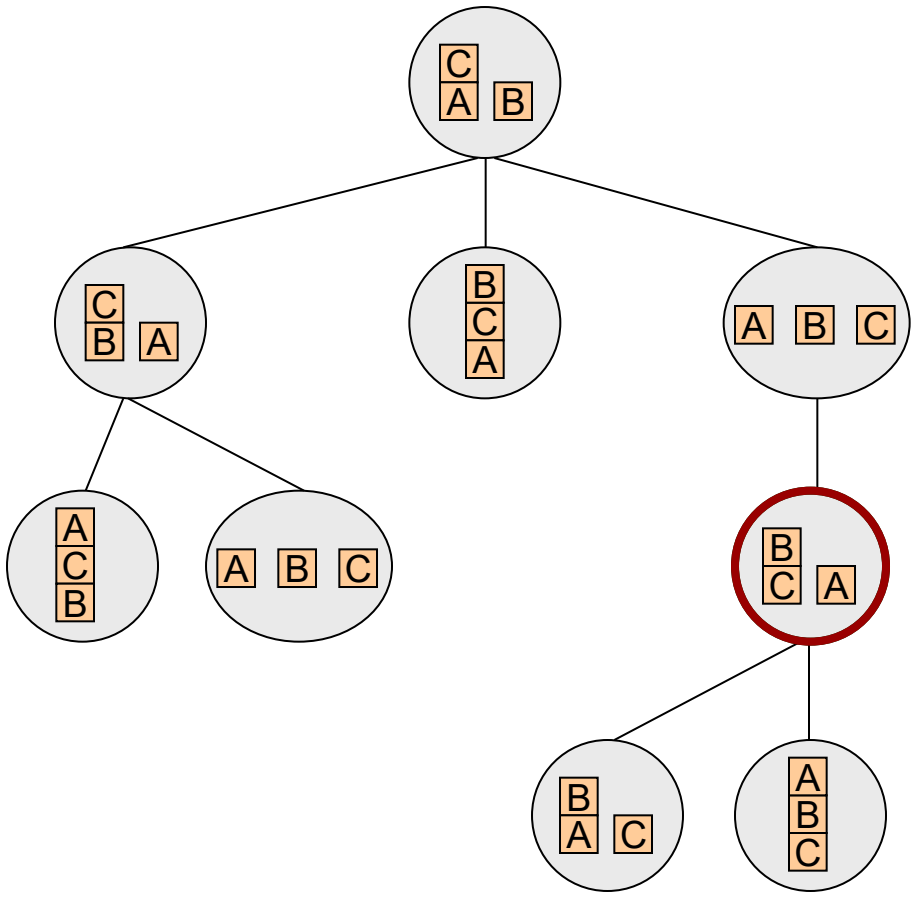
Método de búsqueda:

- *estrategia* para explorar el espacio de estados
- en cada paso se *expande* un estado
- se desarrolla sucesivamente un *árbol de búsqueda*

Método general de búsqueda:

1. seleccionar nodo hoja
2. comprobar si es nodo meta
3. *expandir* este nodo hoja

Arbol de búsqueda:



Algoritmo de búsqueda

Elementos del algoritmo

- el árbol se representa en base a un registro del tipo *nodo*
- *abierta* es una lista de nodos, con las hojas actuales del árbol
- *vacía?* determina si una lista es vacía
- *primero* quita el primer elemento de una lista
- *ordInsertar* añade un nodo a una lista, clasificado según una función de orden

{búsqueda general}
abierta ← s_0
Repetir
  Si *vacía?*(abierta) **entonces**
    devolver(negativo)
  nodo ← primero(abierta)
  Si *meta?*(nodo) **entonces**
    devolver(nodo)
  sucesores ← *expandir*(nodo)
  Para cada n∈sucesores **hacer**
    n.padre ← nodo
    ordInsertar(n,abierta,<orden>)
  Fin {repetir}

Estados repetidos

Problema:

- el mismo estado puede repetirse varias veces en el árbol de búsqueda
- puede generarse el mismo subárbol varias veces

Soluciones:

- ignorarlo
- evitar *ciclos simples*:
  - no añadir el *padre* de un nodo al conjunto de sucesores
- evitar *ciclos generales*:
  - no añadir un *antecesor* de un nodo al conjunto de sucesores
- evitar *todos los estados repetidos*:
  - no añadir *ningún nodo existente* en el árbol al conjunto de sucesores

Clasificación de métodos

Características:

- *Completitud*: se encuentra una solución si existe
- *Optimalidad:* se encuentra la *mejor* solución si hay varias
- *Complejidad en tiempo*: ¿cuánto se tarda en encontrar la solución?
- *Complejidad en espacio*: ¿cuánta memoria se utiliza en la búsqueda?

Tipos de métodos de búsqueda:

- *No informados* (Tema 2)
  - utilizan sólo los conocimientos mínimos (*búsqueda en amplitud*, *búsqueda de coste uniforme*, …)
- *Heurísticos* (Temas 3-5)
  - además utilizan información *aproximada*, y *específica* del problema, para guiar la búsqueda (*Algoritmo A** y extensiones, *búsqueda multiagente*)
- *Con estados estructurados* (Tema 6)
  - Se aprovechan de *características* de los estados para combatir la complejidad (*Satisfacción de restricciones*, …)

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

- Espacio de estados
- Algoritmos de búsqueda no informados

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

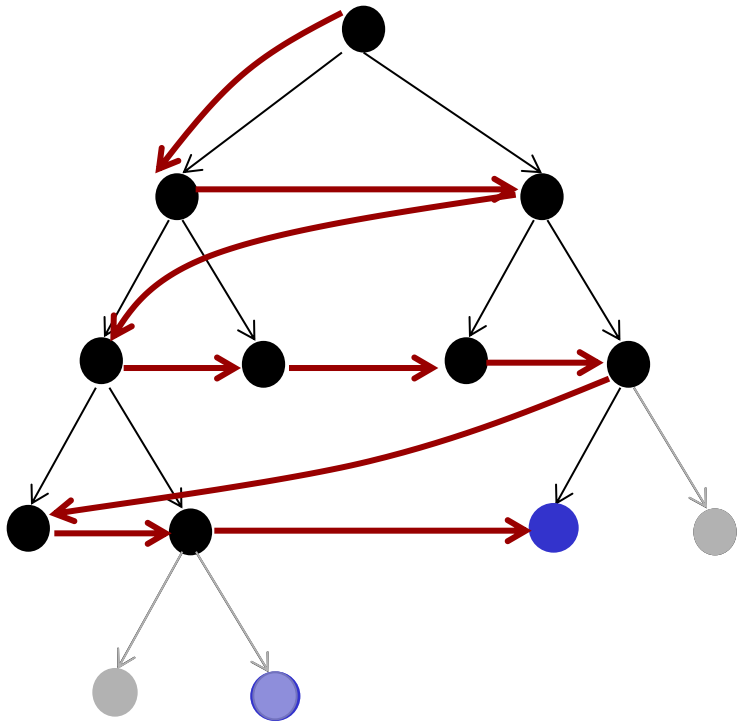
Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Búsqueda en amplitud

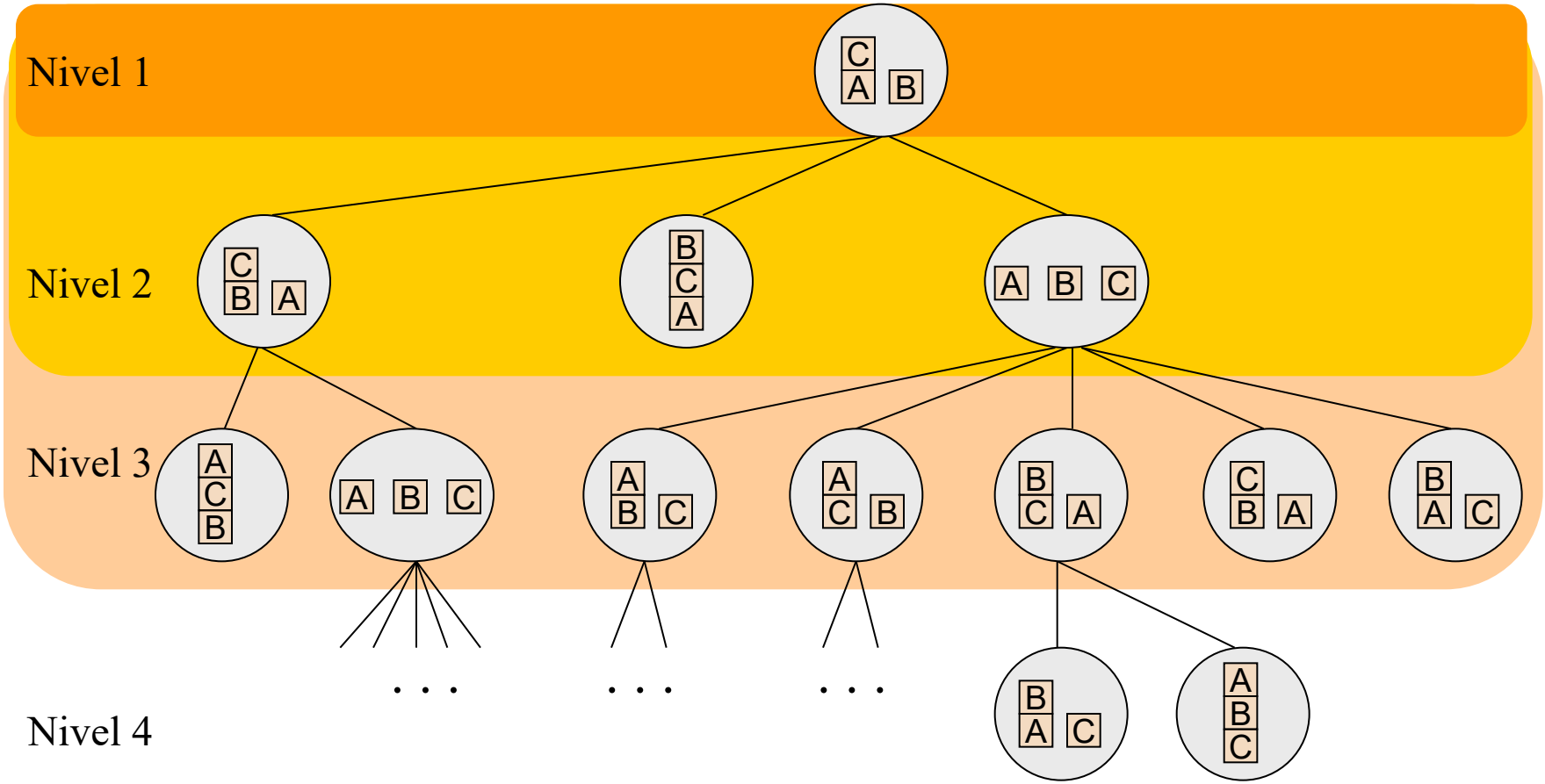
Búsqueda en amplitud:

- inglés: breadth first search
- Estrategia:
  - generar el árbol por niveles de profundidad
  - expandir todos los nodos de nivel *i*, antes de expandir nodos de nivel *i*+1
- Resultado:
  - considera primero todos los caminos de longitud 1, después los caminos de longitud 2, etc.
  - Se encuentra el estado meta de *menor profundidad*



Árbol de búsqueda en amplitud

Búsqueda en amplitud
(evitando ciclos simples):



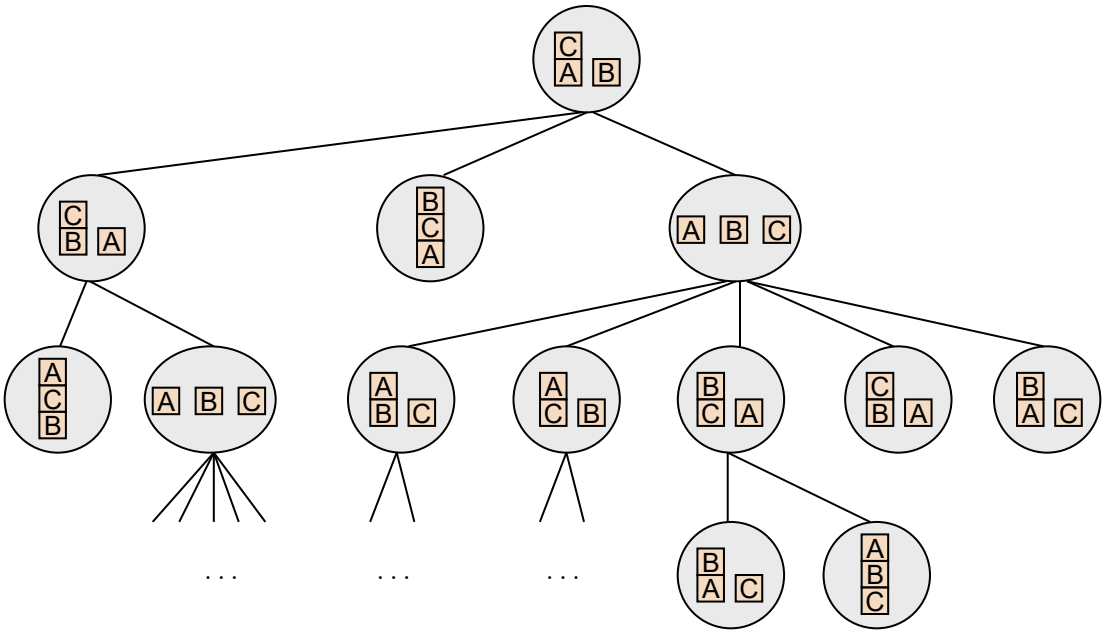
Algoritmo para búsqueda en amplitud

Algoritmo:

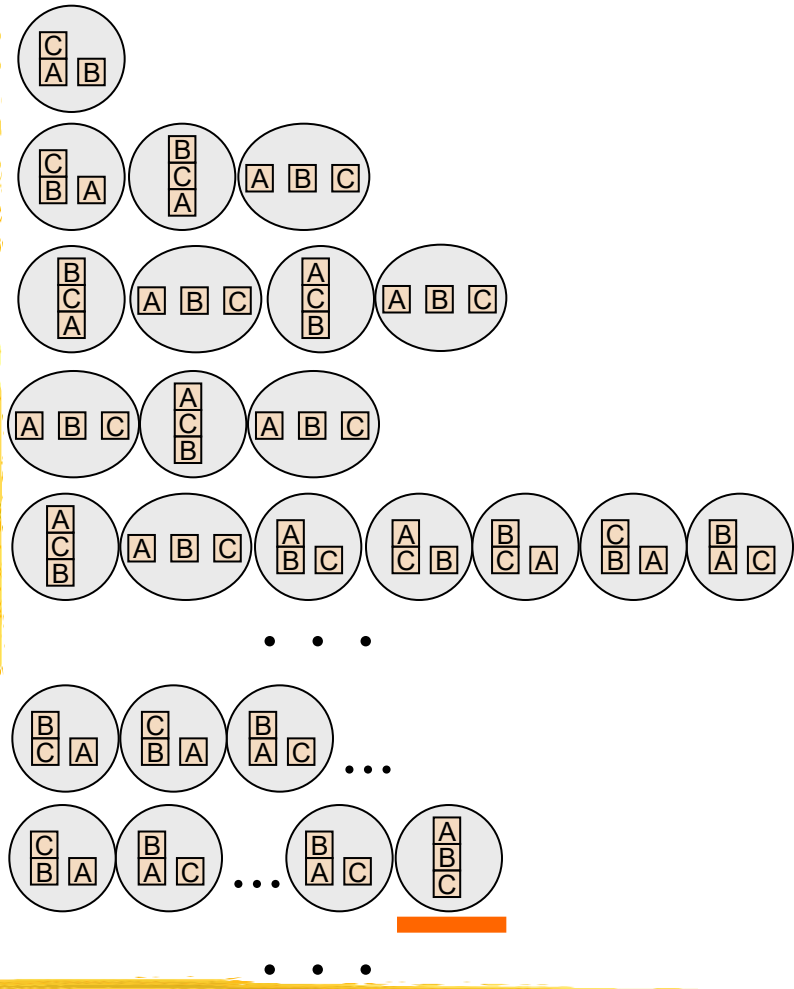
- usar el algoritmo general de búsqueda
- añadir nuevos sucesores al *final* de la lista *abierta*
- *abierta* funciona como *cola*
  - inserción al final
  - recuperación desde la cabeza
- estructura FIFO:
  - siempre expandir primero el nodo más antiguo (es decir: menos profundo)

{búsqueda en amplitud}
abierta ← s_0
Repetir
  Si *vacía?*(abierta) **entonces**
    devolver(negativo)
  nodo ← primero(abierta)
  Si *meta?*(nodo) **entonces**
    devolver(nodo)
  sucesores ← *expandir*(nodo)
  Para cada n∈ sucesores **hacer**
    n.padre ← nodo
    ordInsertar(n,abierta,*final*)
  Fin {repetir}

Árbol de búsqueda en amplitud



Lista *abierta*:



Complejidad

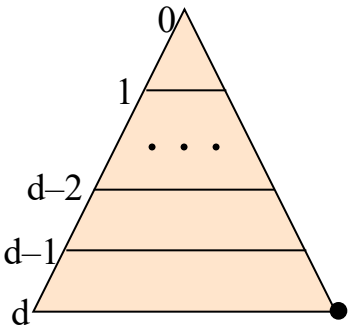
Complejidad en tiempo y espacio:

- proporcional al número de nodos expandidos

Suponemos que en el árbol de búsqueda

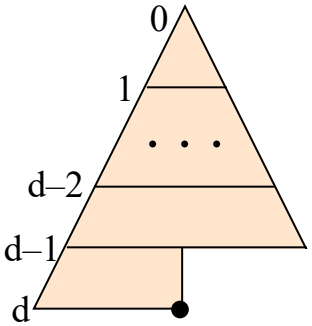
- el factor de ramificación es b
- el mejor nodo meta tiene profundidad d

Peor caso



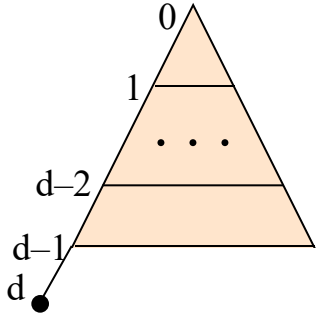
$1+b+\ldots+b^{d-1}+b^{d} \in O(b^{d})$

Caso medio



$1+b+\ldots+b^{d-1}+b^{d}/2 \in O(b^{d})$

Mejor caso



$1+b+\ldots+b^{d-1}+1 \in O(b^{d})$

Requerimientos de tiempo y memoria

Ejemplo: recursos requeridos por la búsqueda en amplitud en el *peor caso*

- factor de ramificación efectivo: 10
- tiempo: 1.000.000 nodos/segundo
- memoria: 1.000 bytes/nodo

d	nodos	tiempo	memoria
2	110	0,11 ms	107 KB
4	11.110	11 ms	10,6 MB
6	10^6	1,1 s	1 GB
8	10^8	2 min	103 GB
10	10^{10}	3 horas	10 TB
12	10^{12}	13 días	1.000 TB
14	10^{14}	3,5 años	99 PB
16	10^{16}	350 años	10.000 PB

Búsqueda en amplitud: análisis

Ventajas:

- completo:
  - siempre se encuentra un nodo meta si existe
- óptimo (para operadores de coste uno):
  - siempre se encuentra el nodo meta menos profundo

Mejoras

Búsqueda de coste uniforme

Problemas:

- complejidad
  - exponencial incluso en el mejor caso
  - los problemas de espacio son aún más graves que los problemas de tiempo

Búsqueda heurística: A*
(ver tema 3)

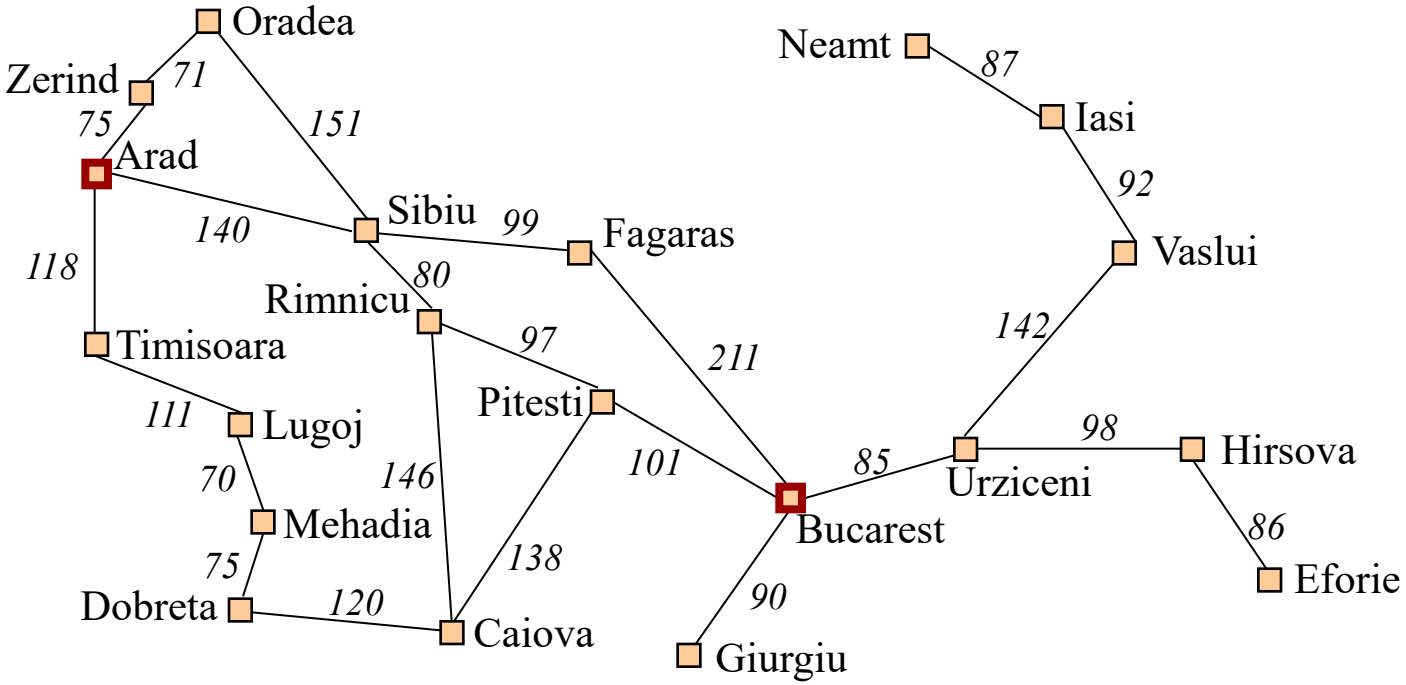
Problema de encontrar rutas

Estado: estancia en una ciudad

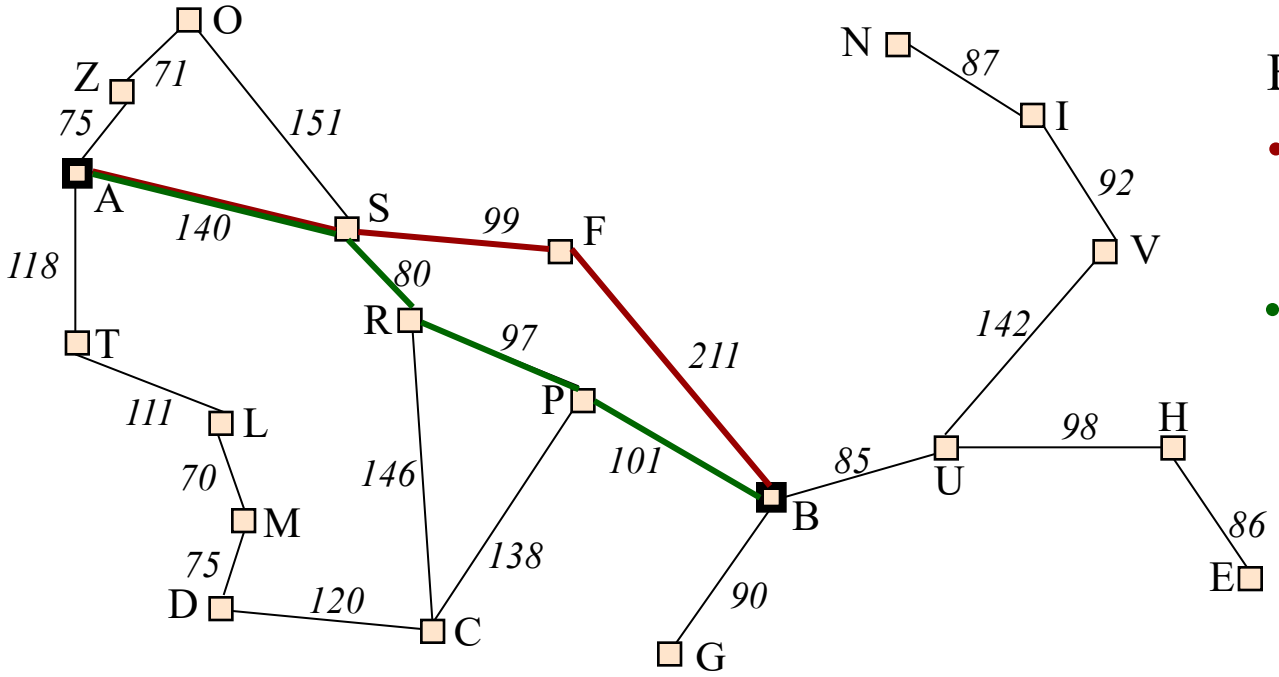
Coste de un operador: distancia por carretera a la ciudad vecina

Operadores: ir a una ciudad vecina

Coste de un plan: suma de distancias entre las ciudades visitadas



Problema de encontrar rutas: ejemplo



Ejemplo:

- p_1 = A-S-F-B
  c(p_1) = 450
- p_2 = A-S-R-P-B
  c(p_2) = 418

Problema:

- La búsqueda en amplitud encuentra el nodo meta de menor profundidad; éste puede *no* ser el nodo meta de coste mínimo
- prof.(B_{p1}) = 3 < 4 = prof.(B_{p2}) / c($p1$) = 450 > 418=c($p2$)

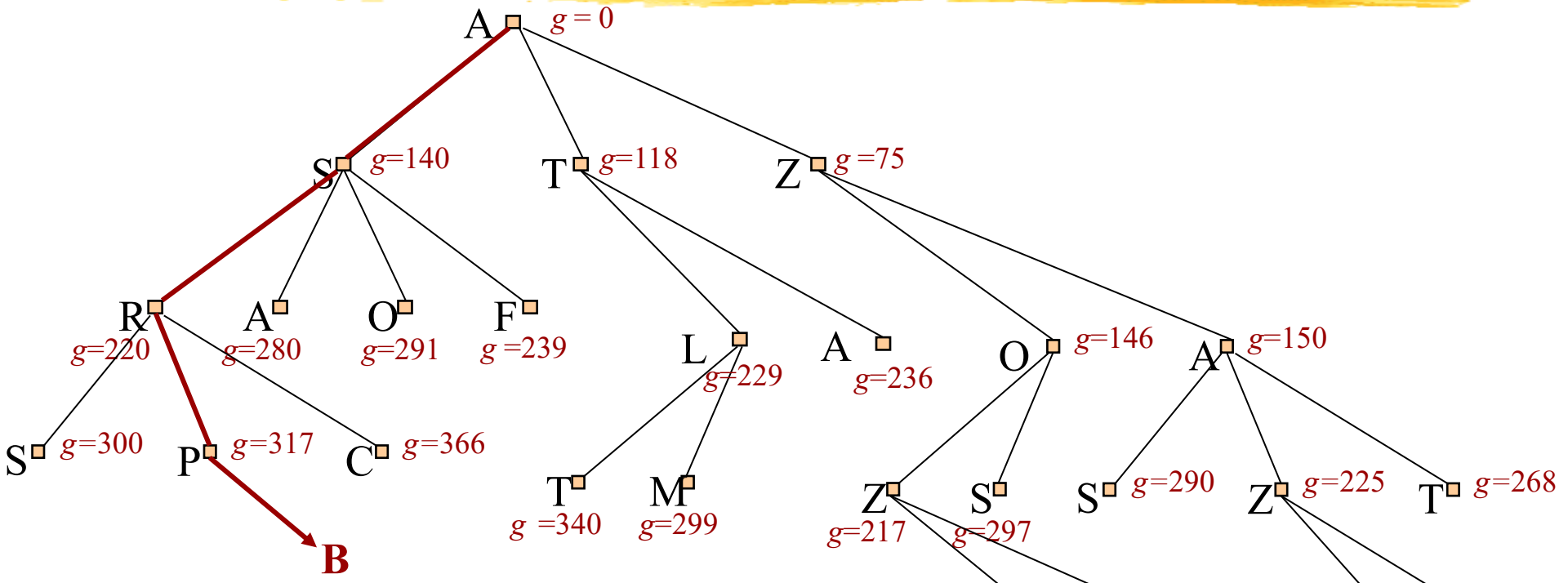
Búsqueda de coste uniforme

Búsqueda de coste uniforme:

- Inglés: uniform cost search
- Idea:
  - guiar la búsqueda por el *coste* de los operadores
- Método:
  - $g(n)$: coste mínimo para llegar del nodo inicial al nodo n
  - expandir siempre el nodo de menor coste g primero
- Algoritmo:
  - almacenar cada nodo con su valor g
  - insertar los nuevos nodos en *abierta* en orden ascendente según su valor g

```
{búsqueda de coste uniforme}
abierta ← s0
Repetir
    Si vacío?(abierta) entonces
        devolver(negativo)
    nodo ← primero(abierta)
    Si meta?(nodo) entonces
        devolver(nodo)
    sucesores ← expandir(nodo)
    Para cada n∈ sucesores hacer
        n.padre ← nodo
        ordInsertar(n,abierta,g)
    Fin {repetir}
```

Ejemplo: Búsqueda de coste uniforme

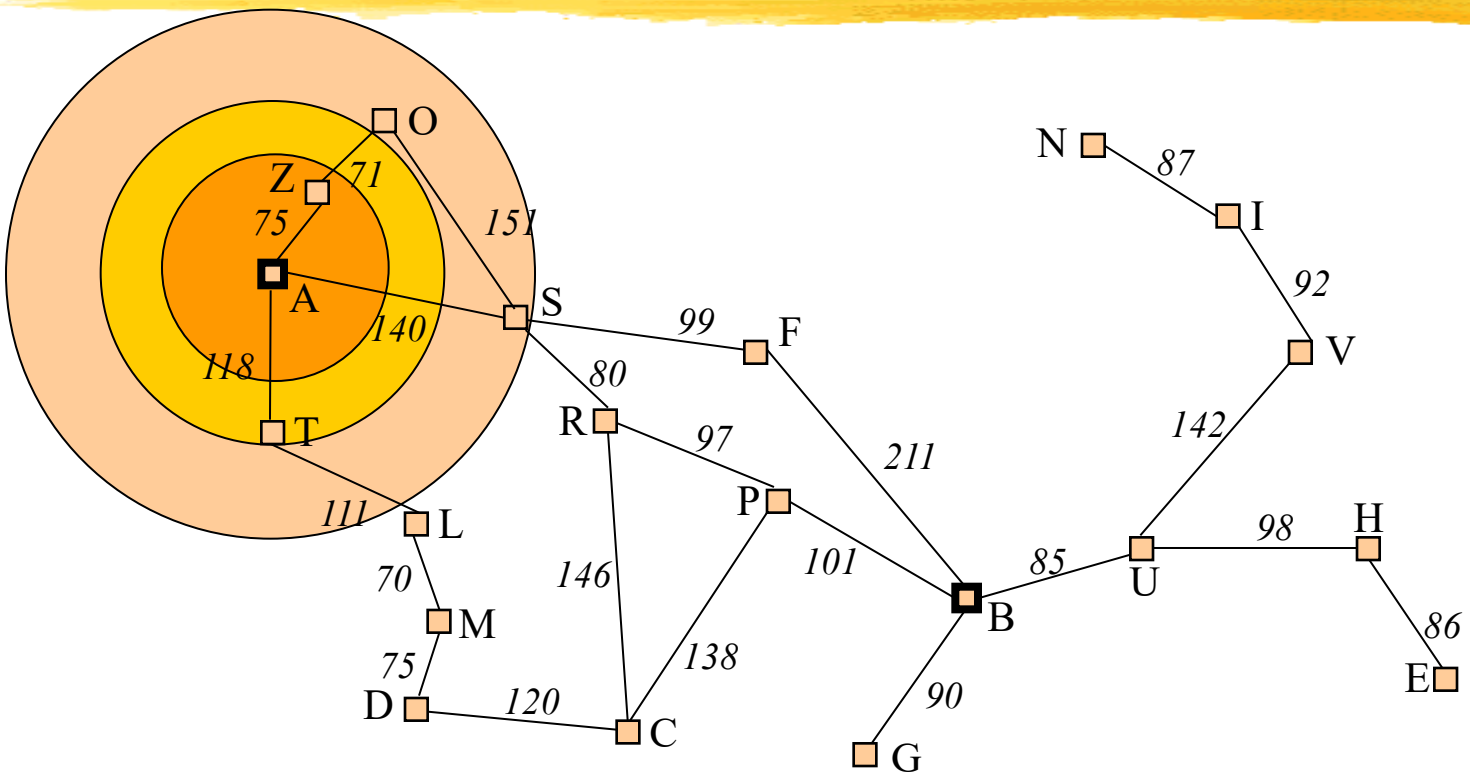


Lista *abierta*:

1. (A,0)
2. **(Z,75), (T,118), (S,140)**
3. (T,118), (S,140), **(O,146), (A,150)**
4. (S,140), (O,146), (A,150), **(L,229), (A,236)**
5. (O,146), (A,150), **(R,220)**, (L,229), (A,236), **(F,239), (A,280), (O,291)**

...

Lógica de la búsqueda de coste uniforme



$g = 80$ $g = 120$ $g = 160$

Características de la búsqueda de coste uniforme

Análisis:

- La búsqueda de coste uniforme enumera sucesivamente todos los nodos del espacio de estados por costes (valores de *g*) *crecientes*
  - la sucesión de valores de *g* *crece de forma monótona* en todos los caminos del árbol de búsqueda, ya que el coste de cualquiera de los operadores es positivo
- La búsqueda de coste uniforme es *completa*:
  - al ser los costes *números enteros* positivos, la sucesión de valores de *g* *no es acotada*
  - por tanto, si un nodo meta existe en el espacio de estados, será expandido alguna vez
- La búsqueda de coste uniforme es *óptima*:
  - al expandir un nodo cualquiera, todos los nodos de menor valor de *g* han sido expandidos *antes*
  - el primer *nodo meta* expandido es el nodo meta de *menor valor de g*
  - el nodo meta de menor valor de *g* es el *nodo meta de menor coste*